Distributor Management System

Code structure and technologies — prepared for the server team · Spare-parts distributor (Sri Lanka) · October 2026

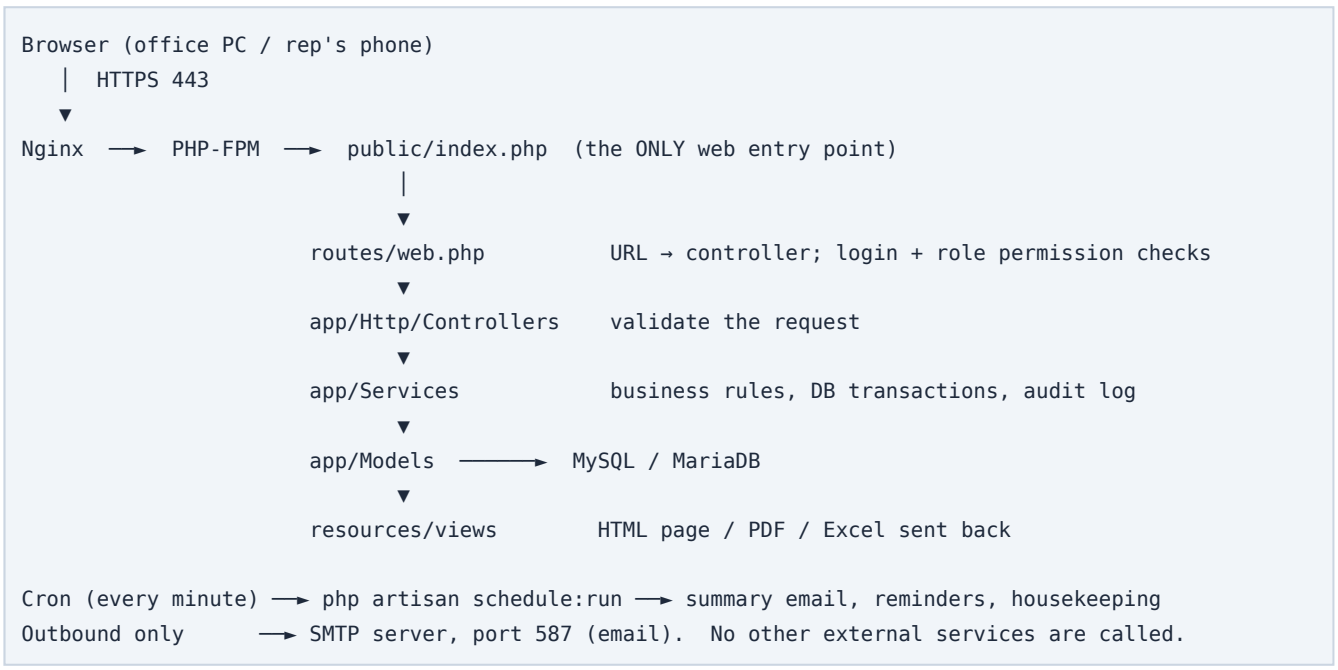
1. In one paragraph

A single **Laravel 13 (PHP)** web application with a **MySQL / MariaDB** database. Pages are built on the server (no separate front-end server, no mobile app — reps use the website on their phones). It needs a web server with **PHP-FPM**, one database, one **cron line** for scheduled tasks, an **SMTP** email account (built in, not yet connected — see section 7), and **HTTPS**. It does **not** need Docker, Redis, Node.js at runtime, or a queue worker. **The system has no built-in backup — the server team must back up the database and uploaded files.**

2. Technologies

Layer	Technology	Notes for the server
Language	PHP 8.3+ (8.4 recommended)	Run with PHP-FPM. Built and tested on 8.5. XAMPP's PHP 8.0 cannot run it.
Framework	Laravel 13.36	Standard Laravel layout and deployment.
Database	MySQL 8.0+ / MariaDB 10.6+	InnoDB, utf8mb4. One database + one user with rights on it only.
Web server	Nginx (or Apache 2.4)	Document root = the `public/` folder.
PHP libraries	Composer 2	dompdf (PDF), openspout (Excel/CSV), picqer barcode generator.
Page templates / UI	Blade, Tailwind CSS 4, Alpine.js 3	CSS/JS are pre-built into `public/build`.
Asset build	Node.js 20+ / Vite 8	**Build time only** (`npm run build`) — not needed while running.
Sessions / cache / queue	Database driver	No Redis or Memcached needed.
Email	SMTP (Laravel Mail)	Password reset, invoices, dealer notices, reminders, daily summary. Needs an SMTP account – see section 7.
Scheduler	Laravel scheduler + cron	`php artisan schedule:run` every minute.
Required PHP extensions	bcmath, ctype, curl, dom, fileinfo, filter, iconv, mbstring, openssl, pdo_mysql, session, SimpleXML, tokenizer, xml, xmlreader, xmlwriter, zip · recommended: opcache, intl · not needed: gd, imagick, redis	

3. How a request flows


Browser (office PC / rep's phone)
HTTPS 443
Nginx
PHP-FPM
public/index.php (the ONLY web entry point)
routes/web.php
URL → controller; login + role permission checks
app/Http/Controllers
validate the request
app/Services
business rules, DB transactions, audit log
app/Models
MySQL / MariaDB
resources/views
HTML page / PDF / Excel sent back
Cron (every minute) → php artisan schedule:run → summary email, reminders, housekeeping
Outbound only → SMTP server, port 587 (email). No other external services are called.

4. Folder structure — what matters on the server

DEPLOY = copy to server **WRITABLE** = web user (e.g. `www-data`) must be able to write **NOT NEEDED** = do not copy / not used at runtime

```
distributor/
├── app/                        DEPLOY      application code (PHP)
│   ├── Console/Commands/                   scheduled jobs: daily summary, overdue reminders, close stale visits
│   ├── Http/Controllers/                   38 controllers — one per screen / module
│   ├── Http/Middleware/                    role / area checks
│   ├── Mail/                               email messages (invoice, payment notice, reset, summary…)
│   ├── Models/                             47 database models
│   ├── Services/                           business logic: stock, sales, payments, purchases, mail…
│   └── Support/                            permissions, totals/tax, settings, export, schedule list
├── bootstrap/                  DEPLOY      framework start-up
│   └── cache/                  WRITABLE    compiled config / routes
├── config/                     DEPLOY      settings (read values from .env)
├── database/                   DEPLOY
│   ├── migrations/                         19 files — create / update all tables (php artisan migrate)
│   └── seeders/                            DatabaseSeeder = live start data · DemoDataSeeder = demo ONLY
├── public/                     DEPLOY      ◄── WEB SERVER DOCUMENT ROOT (nothing above it is web-visible)
│   ├── index.php                           single entry point
│   ├── build/                              compiled CSS / JS (from npm run build)
│   └── images/                             logo
├── resources/                  DEPLOY      Blade page / email / PDF templates (+ CSS/JS sources)
├── routes/                     DEPLOY      web.php (pages) · console.php (schedule)
├── storage/                    WRITABLE    must be writable and must be BACKED UP
│   ├── app/private/                        uploaded payment slips, cheque images, product pictures
│   ├── framework/                          compiled views, cache files
│   └── logs/laravel.log                    application log — set up log rotation
├── tests/                      optional    automated tests (run against a separate test database)
├── vendor/                     DEPLOY      PHP libraries — created by: composer install --no-dev
├── node_modules/               NOT NEEDED  build tools only (64 MB) — never on the server
├── artisan                     DEPLOY      command-line tool (migrate, cache, schedule:run)
├── composer.json / .lock       DEPLOY      PHP library list (exact versions locked)
├── package.json / vite.config.js           only needed where npm run build is done
└── .env                        SERVER      secrets & settings — created on the server, never shared
```

> **Security:** point the web server at `public/` only. If the project root is exposed, `.env` (database and mail passwords) and `storage/` (payment slips) could be downloaded.

Sizes (current)

Part	Size	Grows?
Application code (without vendor / node_modules)	≈ 3 MB	No
vendor/ (PHP libraries)	≈ 80 MB	Only on updates
public/build (CSS/JS)	≈ 0.3 MB	No

Database	≈ 3 MB (demo data)	Yes – with transactions
storage/app/private (uploads, up to 5 MB each)	small now	**Yes – main growth**, plan 5–10 GB / year (estimate)

5. Configuration (.env on the server)

Key	Value / meaning
`APP_ENV` / `APP_DEBUG`	`production` / `false` (never true on the live server)
`APP_URL`	e.g. `https://erp.company.lk` (used in email links)
`APP_KEY`	generated once with `php artisan key:generate` – keep it safe
`APP_TIMEZONE`	`Asia/Colombo`
`DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD`	database connection
`SESSION_DRIVER, CACHE_STORE, QUEUE_CONNECTION`	`database` · plus `SESSION_SECURE_COOKIE=true` on HTTPS
`MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM_ADDRESS`	SMTP account. With `MAIL_MAILER=log` nothing is delivered (written to the log only).
`EMAIL_DEALER_PAYMENT_NOTICES, EMAIL_OVERDUE_REMINDERS, DAILY_SUMMARY_TIME, OVERDUE_REMINDER_TIME`	email options (also editable in Setup → Settings)
`COMPANY_NAME` … `COMPANY_TAX_NUMBER, CURRENCY_SYMBOL`	defaults; normally set in Setup → Settings

6. Command-line entry points

Command	When
`composer install --no-dev --optimize-autoloader`	first install and every update
`npm ci && npm run build`	when CSS/JS changes (can be done on a build PC and `public/build` copied)
`php artisan migrate --force`	first install and every update (creates / updates tables)
`php artisan db:seed --force`	first install only: roles, permissions, banks, holidays, first admin. **Do not** run DemoDataSeeder on live.
`php artisan config:cache · route:cache · view:cache`	after every deployment (speed)
`php artisan down` / `php artisan up`	maintenance window during updates
`* * * * * cd /var/www/distributor && php artisan schedule:run`	cron – permanent, every minute

Scheduled tasks (run by the cron line)

Task	When	What
`summary:daily`	daily 07:30	email of waiting work to managers
`reminders:overdue`	Monday 08:30	overdue-invoice reminder to dealers (only if enabled)
`visits:close-stale`	daily 00:15	closes rep visits left open
`auth:clear-resets`	daily 03:00	deletes expired password-reset links

Setup → Scheduled tasks shows in red when the scheduler is not running.

7. Email (SMTP)

> **Status:** email sending is **built and tested** in the system, but **no SMTP account is connected yet**. The current setting `MAIL_MAILER=log` only writes emails to `storage/logs/laravel.log`, so nothing reaches anyone until the server team adds an SMTP account.

What the system emails

Email	Sent to	When
Password reset link	the user	"Forgot your password?" or admin "Send reset link"
Invoice with PDF attached	the dealer	staff click "Email invoice"
Payment verified / rejected, cheque returned	the dealer (if it has an email)	when the accountant acts (can be switched off in Settings)
Overdue invoice reminder	dealers with overdue invoices	Mondays 08:30 (off by default)
Daily summary (counts of waiting work)	managers with the permission	daily 07:30
Test email	any address	Setup → Scheduled tasks → Send test

Volume is low (tens to a few hundred emails per day), so any business mail provider is enough. Emails are sent immediately; no queue worker is needed.

What the server team must provide

- A dedicated sending mailbox, e.g. `system@company.lk` or `noreply@company.lk` (not a personal mailbox).
- SMTP host, port, username and password (an **app password** or SMTP relay for Google / Microsoft).
- **SPF** and **DKIM** DNS records for the sending domain (and preferably **DMARC**), so dealer emails do not land in spam.
- Outbound firewall access from the server to the SMTP host on port **587** (or 465).

Common providers

Provider	Host	Port	Notes
Google Workspace	`smtp.gmail.com`	587 (TLS)	Needs 2-step verification + app password, or use the Workspace SMTP relay
Microsoft 365	`smtp.office365.com`	587 (TLS)	SMTP AUTH must be enabled for the mailbox
Amazon SES	`email-smtp.<region>.amazonaws.com`	587 (TLS)	Verify the domain; request production access

Hosting company / cPanel mail	as given by the host	587 or 465	Check daily sending limits

Check the exact host and port with your provider; these are the usual published values.

Where to enter the SMTP details

Option A – in the system (recommended): a Super Admin opens **Setup → Email settings**, enters host, port, encryption, username, password and the from-address, saves, then clicks **Test connection** and **Send test email**. The password is stored **encrypted** with the application key (`APP_KEY`) – if APP_KEY is ever changed, re-enter the password. Settings saved here override `.env`.

Option B – in .env (used when nothing is saved in Email settings):

Settings in .env

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.yourprovider.com
MAIL_PORT=587
MAIL_USERNAME=system@company.lk
MAIL_PASSWORD=<app password — entered on the server only>
MAIL_FROM_ADDRESS=system@company.lk
MAIL_FROM_NAME="Company name"
```

Then run `php artisan config:clear` (or `config:cache` on production), open **Setup → Scheduled tasks**, click **Send test** and confirm the email arrives. Every email and its result (sent / failed with the reason / not delivered) is listed in **Setup → Email log**. Dealer replies go to the company email set in Setup → Settings.

8. Database

19 migration files create 56 tables. Main groups:

Area	Tables
Catalogue	categories, brands, products, product_part_numbers, product_fitments
Stock	warehouses, stock_levels, stock_movements, shipments, shipment_items
Purchasing	suppliers, purchase_orders(+items), goods_receipts(+items), supplier_bills, supplier_payments(+allocations)
Sales	customers (dealers), quotations(+items), sales_orders(+items), invoices(+items), credit_notes(+items)
Payments	payments, payment_allocations, payment_submissions, payment_status_history, document_attachments
Banking	banks (30 CBSL-licensed banks), bank_branches, company_bank_accounts
Users & control	users, roles, permissions, permission_role, rep_visits, audit_logs, settings
Communication	email_logs, task_runs, notices, holidays
Laravel	sessions, cache, cache_locks, jobs, failed_jobs, password_reset_tokens, migrations

> **Backup is the server team's job:** daily `mysqldump --single-transaction` of the database + copy of `storage/app` + the `.env` file, stored off the server, with a restore tested regularly.

9. Application code by module

Module	Controllers	Services (business logic)
Catalogue & stock	Product, Lookup (categories/brands), Stock, Warehouse, Shipment	StockService (only place stock changes), ShipmentService
Purchasing	Supplier, PurchaseOrder, GoodsReceipt, SupplierBill, SupplierPayment	PurchaseService, PayablesService
Sales	Customer, Quotation, SalesOrder, Invoice, CreditNote, DocumentPdf, InvoiceEmail	SalesService, QuotationService
Payments & banks	Payment, PaymentInbox, PaymentVerification, Cheque, Bank, CompanyBankAccount	PaymentService, PaymentInboxService, DealerNotifier
Field sales	Rep (rep dashboard), Field (live rep monitor)	—
Finance & reports	Expense, Report, Dashboard	—
Data	Import (Excel / CSV)	ImportService

Administration	User, Role, Settings, Auth, PasswordReset, System (tasks, email log), Notice (notices, holidays)	MailService (all email, logged)

Security built into the code

- 12 roles with 63 permissions, checked on every route; reps see only their assigned dealers.
- Passwords hashed (bcrypt); login and password reset throttled; CSRF protection on every form.
- Uploaded files stored privately in `storage/` and served only to logged-in users with permission.
- Money and stock changes run in database transactions; corrections are reversals, never deletions; audit log of sensitive actions.

Automated tests

45 feature tests (PHPUnit) in `tests/Feature`: access control, banks, catalogue & shipments, payment verification, email & scheduler, dashboard, and a smoke test that opens every page. They use a **separate** database (`distributor_test`) and must never point at the live database.

> **Not built (need accounts / credentials):** WhatsApp messages and reading payment slips from an email inbox. Backups are intentionally not part of the system.